

LLM-Assisted Security Vulnerability Analysis for Educational Websites: Risk Identification via LLM-EduAttackGraph

Chao Liu¹, Member, IEEE, Jiaxing Liu², Boxi Chen³, Student Member, IEEE, Daxin Zhu⁴,
Ching-Chun Chang⁵, Member, IEEE, and Chin-Chen Chang⁶, Fellow, IEEE

Abstract—The digital transformation of educational systems has significantly optimized administrative workflows and enhanced the user experience for educators and learners. However, the accumulation of sensitive personal data on educational websites has made them prime targets for cyber threats. Despite growing awareness of these security challenges, the technical roots of vulnerabilities within such platforms remain insufficiently explored. To address this gap, we introduce LLM-EduAttackGraph, a specialized tool designed to assist in vulnerability detection by leveraging large language models (LLMs). Rather than serving as a fully automated monitoring system, LLM-EduAttackGraph operates as a human-in-the-loop assistant, combining expert knowledge with the analytical capabilities of LLMs to help identify potential penetration paths based on network fingerprint information. Using LLM-EduAttackGraph, we have so far identified 961 penetration vulnerabilities across educational websites in mainland China—a number that continues to grow as analysis progresses. These findings demonstrate the tool’s practical value in augmenting cybersecurity research and efforts. Our in-depth analysis of the discovered vulnerabilities reveals that limited developer experience and a heavy dependence on outsourced website development are key contributing factors. By shedding light on these root causes, our research offers actionable strategies and insights aimed at improving the cybersecurity posture of educational platforms and ensuring the sustainable development of online education. Furthermore, we have compared LLM-

EduAttackGraph with several existing large model penetration tools to demonstrate the performance of LLM-EduAttackGraph. Such strengths include its low demand for hardware resources and having undergone empirical verification.

Index Terms—Cybersecurity, educational websites, LLM-EduAttackGraph, sensitive personal data.

I. INTRODUCTION

ILLLEGAL actors have various motives for attacking networks, such as acquiring and selling private information for profit. Cyberattacks may sometimes have political and military intentions [23]. In the medical field, personal medical information (including patient full names, dates of birth, detailed diagnostic reports, specific disease and treatment histories, medication details, and records of genetic diseases within the family, among other key data) is considered sensitive. The leakage of personal medical information threatens the personal dignity of patients and basic rights such as individual rights [5]. In the manufacturing sector, manufacturing machines are increasingly equipped with sensors and connected via wireless networks or wired Ethernet, making digital manufacturing systems more vulnerable to cyberattacks [37].

Educational websites refer to the official websites of primary and secondary schools, universities, and education bureaus. Educational websites primarily serve students, teachers, and other personnel. As an indispensable part of the modern educational system, educational websites are a reflection of the digitization and modernization of education. The spread of COVID-19 has accelerated the transition of the higher education system from offline to online [20], advancing the process of digitalization of higher education. Educational websites store a large amount of personal information and other core data, which tempt hackers to target higher education institutions as primary targets [17]. In recent years, there has been an increasing number of information leakage incidents in higher education institutions. Domestically, more than 20 000 students’ information at Zhengzhou Sias University [40] was leaked, and tens of thousands of teachers’ and students’ information from a university in Jiangxi [25] was sold abroad. Internationally, universities such as Stanford University [10], Queensland University of Technology [4], and Western Sydney University [3] have all experienced varying degrees of data breaches. Information leakage not only brings direct troubles and risks to the users of the affected institutions but also causes serious damage to the reputation and trustworthiness of educational institutions. Therefore, the data information

Received 15 August 2025; revised 14 October 2025; accepted 7 November 2025. Date of publication 19 November 2025; date of current version 8 January 2026. This work was supported in part by the 2025 Xiamen Overseas Returnee Research Project under the Grant titled, in part by the Fujian Provincial Key Technological Innovation Research & Industrialization Project under Grant 2024XQ017, in part by the Sixth batch of Quanzhou City’s introduction of high-level talent team projects 2, in part by the Natural Science Foundation of Fujian Provincial Science and Technology Department under Grant 2023H6034, and in part by the 2025 College Student Innovation and Entrepreneurship Training Program of Xiamen Medical College under Project 202512631013 and Project 202512631034. (Chao Liu, Jiaxing Liu, and Boxi Chen contributed equally to this work.) (Corresponding authors: Daxin Zhu; Chin-Chen Chang.)

Chao Liu is with the Department of Public Health and Medical Technology, Xiamen Medical College, Xiamen 361023, China.

Jiaxing Liu is with School of Advanced Manufacturing, Fuzhou University, Quanzhou 362251, China.

Boxi Chen is with Faculty of Applied Sciences, Macao Polytechnic University, Macao, China

Daxin Zhu is with the School of Mathematics and Computer Science, Fujian Provincial Key Laboratory of Data-Intensive Computing, Fujian University Laboratory of Intelligent Computing and Information Processing, Quanzhou Normal University, Quanzhou 362046, China (e-mail: dex@qztc.edu.cn).

Ching-Chun Chang is with the National Institute of Informatics, Tokyo 101-8430, Japan.

Chin-Chen Chang is with Feng Chia University, Taichung 40724, Taiwan (e-mail: ccc@o365.fcu.edu.tw).

Digital Object Identifier 10.1109/IJOT.2025.3631562

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

security of educational websites, especially universities, has become an issue that requires in-depth research.

Despite ongoing advancements in cybersecurity, existing penetration testing methods for educational websites remain largely reliant on the personal expertise and intuition of security professionals. These manual approaches often involve repetitive, time-consuming, and tedious tasks such as fingerprint collection, vulnerability matching, and exploitation path analysis. The reliance on human judgment introduces inconsistencies in testing quality and limits scalability, especially when auditing a large number of websites. Moreover, many existing vulnerability detection systems are either too rigid—relying on rule-based engines with limited adaptability—or too generic, lacking contextual understanding of educational platforms' unique structural and security characteristics. These systems struggle to efficiently integrate historical vulnerability data, interpret nuanced attack surfaces, or generate meaningful insights beyond surface-level scanning. As a result, analysts may overlook latent vulnerabilities or spend disproportionate effort on low-risk components.

We adopted a coordinated vulnerability disclosure approach to conduct penetration testing on university websites, including those of first- and second-level colleges within universities, as well as websites of education bureaus in selected regions of mainland China. While our testing focused exclusively on institutions within mainland China due to legal and ethical constraints, we also discuss the broader applicability of our findings to educational websites in other countries in Section V. In total, we identified more than 961 vulnerabilities across educational websites. Among these, 39.2% were found in teaching-oriented universities offering only undergraduate programs, while the remaining 60.8% were distributed across other types of educational institutions. The sample also includes several highly recognized universities in mainland China. To support the testing process, we developed LLM-EduAttackGraph, a practical and effective tool that integrates large language models (LLMs) to assist in identifying potential vulnerabilities. The process begins with website fingerprint scanning, which collects structural and component-level metadata. These fingerprints are then fed into the core of a pretrained LLM-based system, which, by referencing historical vulnerability data, infers likely penetration paths and potential weaknesses. Rather than performing full automation, LLM-EduAttackGraph operates in a human-in-the-loop manner to support expert analysis. As illustrated in Fig. 1, **LLM-EduAttackGraph** is designed to assist in penetration testing across six major categories of vulnerabilities commonly found in educational websites: ① remote command/code execution; ② SQL injection; ③ weak passwords; ④ unauthorized access; ⑤ token tampering; and ⑥ sensitive information disclosure. These six categories correspond to different attack surfaces within the architecture of educational web systems. For instance, SQL injection primarily targets the database layer, exploiting improper input handling in query construction. Remote code execution (RCE) typically affects the application layer, enabling attackers to execute arbitrary commands on the server. Weak passwords and unauthorized access are commonly associated with authentication and access con-

trol components, while token tampering compromises session management mechanisms. Finally, sensitive information disclosure often results from misconfigurations or insecure data storage practices. Throughout the testing, we strictly adhered to principles of legality and compliance, ensuring that all activities were authorized. Discovered vulnerabilities were promptly reported to the appropriate institutions along with mitigation recommendations. Based on our analysis of real-world security flaws, we observed the following major issues.

- 1) Many systems suffer from unpatched components or outdated frameworks, leaving them vulnerable to known exploits.
- 2) Numerous educational institutions outsource website development to third-party vendors or rely heavily on pre-built frameworks. In many cases, default credentials were not changed post-deployment—either due to oversight by the development company or a lack of cybersecurity expertise within the institution—leading to common access vulnerabilities across multiple sites developed by the same vendor.
- 3) Core digital assets are more prone to logical vulnerabilities and vertical privilege escalation, whereas peripheral systems often exhibit weak password issues.
- 4) A school's academic prestige does not necessarily correlate with its cybersecurity maturity. Even institutions with strong reputations are vulnerable to data leakage and other critical risks.

LLM-EduAttackGraph's core design philosophy lies in a decoupled architecture of *offline knowledge base + online retrieval and reasoning*. Currently, there are several LLM-based automated penetration testing models in the field of cybersecurity. PentestGPT has effectively validated the application potential and technical feasibility of LLMs in automated penetration testing tasks by designing a multistage execution process covering reasoning, generation, and parsing modules, and integrating experimental verification [12]. AutoAttacker has achieved a high post-penetration attack success rate in simulated enterprise network environments by constructing modular agents and introducing the RAG mechanism based on experience reuse [39]. MAPTA, as a model specifically developed for web application penetration testing, relies on a multiagent architecture design and an end-to-end proof-of-concept (PoC) execution mechanism, which effectively reduces interfering information during the vulnerability detection process [11].

However, the aforementioned models have limitations and fail to be directly adapted to the needs of educational network scenarios. First, like the comprehensive penetration framework of AutoAttacker, the complex multiagent interaction and real-time tool calling lead to extremely high instantaneous GPU/CPU resource consumption, making large-scale deployment in educational institutions with limited computing power unfeasible. Second, these models are originally designed for end-to-end attack chain verification of vulnerabilities, and their inherent offensive nature carries abuse risks, which may easily trigger security alerts in campus networks with strict compliance requirements. Third, the internal state transitions and decision-making processes

TABLE I
MODEL ATTRIBUTE COMPARISON—○ DENOTES FULL SUPPORT AND ● DENOTES SUPPORT

Model	Designed for Web Penetration	Model Application Poses No Direct Risks	Low Hardware Resource Consumption	Model Decisions Traceable	Higher Update Timeliness	Empirical Support for Experimental Results
LLM-EduAttackGraph	●	●	●	●	●	●
PentestGPT [12]	○	○	●	○	○	○
AutoAttacker [39]	○	○	●	○	○	○
MAPTA [11]	●	○	●	●	○	○

of multiagent systems are akin to a *black box*, resulting in poor interpretability and hindering administrators from conducting audits, reviews, and trust analysis of results. In contrast, LLM-EduAttackGraph does not pursue fully automated attacks; instead, it is positioned as an intelligent auxiliary analysis system, which maintains high targeting while achieving low resource consumption, inherent defense orientation, and a clear, traceable decision-making process.

The differences in key attributes between LLM-EduAttackGraph, PentestGPT [12], AutoAttacker [39], and MAPTA [11] are presented in Table I. The attributes of LLM-EduAttackGraph are as follows:

Designed Specifically for Web Penetration: LLM-EduAttackGraph employs a Web application-oriented security analysis paradigm. Its core components—fingerprint scanning and knowledge base retrieval—are all constructed around the core characteristics of Web assets (such as HTTP/S services, open ports, URL structures, and software identifiers, among others).

The Application of LLM-EduAttackGraph Does Not Directly Pose Risks to the Web: LLM-EduAttackGraph workflow terminates at the generation of vulnerability analysis and remediation recommendations and does not involve the execution of actual exploit payloads. Such a purely defensive positioning eliminates the possibility of the tool being abused.

Facilitating Large-Scale Deployment With Controllable Costs: Benefiting from the decoupled architecture of *offline knowledge base construction + online lightweight reasoning*, LLM-EduAttackGraph pre-positions the computationally intensive vectorization process. Consequently, the online services can operate with only minimal computing resources.

High Interpretability of the Decision-Making Process: The entire analytical chain of LLM-EduAttackGraph is clear and transparent. Security administrators can easily trace back which fingerprint features triggered the retrieval of specific historical vulnerabilities and understand the context on which the LLM generates remediation recommendations. This interpretability greatly enhances the credibility of the results.

Superior Real-Time Knowledge Updating: LLM-EduAttackGraph constructs a vector database from continuously updated historical vulnerability repositories (e.g., Awesome-POC), which enables the timely integration of the latest vulnerability knowledge. In contrast, models that require calling external static tools are still constrained by explicit update mechanisms. Their synchronization cycles are typically significantly longer than the response time of the models' internal knowledge, resulting in detection lag when confronting newly disclosed vulnerabilities.

Real-World Validated Effectiveness: LLM-EduAttackGraph is not confined to the theoretical or laboratory stage; instead,

it has undergone extensive testing in the real-world environments of Chinese educational websites and achieved specific, quantifiable outcomes, which confirms its effectiveness.

Contributions: To summarize, our contributions are as follows.

- 1) We propose **LLM-EduAttackGraph**, a novel human-in-the-loop penetration testing framework that integrates fingerprint scanning with LLMs to assist in the discovery and reasoning of vulnerabilities in educational websites. The system automates the fingerprint extraction phase and leverages pretrained LLMs to analyze structural features and historical patterns, generating potential attack paths and identifying likely vulnerabilities. This approach addresses the limitations of traditional expert-driven methods, reducing repetitive workload while improving coverage and scalability.
- 2) Utilizing **LLM-EduAttackGraph**, we conducted a large-scale penetration testing study across educational websites in mainland China and uncovered over 961 real-world vulnerabilities. All issues were responsibly disclosed to the corresponding institutions, many of which expressed appreciation and formally recognized our contributions. The effectiveness of our system in identifying practical security issues demonstrates its utility in real-world cybersecurity applications.
- 3) Based on the vulnerabilities identified, we performed a detailed analysis revealing several systemic problems. We observed that core digital assets are more prone to logical flaws and privilege escalation, while peripheral assets commonly suffer from weak password configurations. Additionally, many vulnerabilities stem from insufficient in-house development expertise and reliance on third-party vendors who reuse insecure components across multiple websites. We offer targeted recommendations to help educational institutions mitigate these root causes and strengthen their security posture.
- 4) We have compared **LLM-EduAttackGraph** with several existing large model penetration tools to demonstrate its performance. Such advantages of LLM-EduAttackGraph include: being specifically designed for Web penetration, posing no direct threat to the Web when the model is applied, having low demand for hardware resources, model decisions are traceable, the model's knowledge base being updated quickly, and having undergone empirical verification.

II. RELATED WORK

Research on addressing information leakage in educational websites has been extensive. Bongiovanni systematically synthesized studies on information security in higher education

institutions, identifying key concepts and patterns over 13 years of research, providing theoretical insights into the issue of information leakage in educational environments [8]. Li et al. [21], using the crime opportunity theory and routine activity theory, analyzed several key factors affecting data security risks, such as system vulnerabilities, cross-border data transmission, institutional academic capabilities, and cloud storage adoption. Employing statistical methods like Poisson regression, they systematically evaluated the impact of these factors on Chinese universities [21]. Dolliver et al. [14] analyzed intrusion detection data from a U.S. university, offering theoretical foundations and practical guidance for educational website security strategies. Omeiza and Owusu-Tweneboah [28] conducted penetration testing at Carnegie Mellon University Africa using BurpSuite, identifying and categorizing vulnerabilities by severity and proposing corresponding mitigation measures.

Sun et al. [35] proposed a packet content-oriented detection model for RCE attacks, focusing on XML external entity (XXE), expression language injection (ELi), and insecure deserialization (IDSER). Their model integrates static analysis and machine learning to improve detection precision. Zheng and Zhang [41] introduced a context-sensitive interprocedural analysis method for identifying RCE vulnerabilities, efficiently addressing dynamic file inclusion and cross-request dependencies. Xiao et al. [38] developed XGuard, a defense technology leveraging static analysis to extract sensitive application programming interface (API) information from cross-platform applications, defining restrictive configurations to mitigate Xrce attacks.

For structured query language (SQL) injection vulnerabilities, Bayyapu [6] summarized various mitigation strategies, including static and dynamic analysis, taint-based detection, and query development paradigms. Muduli et al. [26] proposed two CNN models, SIDNet-1 and SIDNet-2, to classify SQL injection attacks, addressing overfitting issues with dropout layers in SIDNet-2. Traditional machine learning methods, such as random forest classifiers combined with honeypot servers, have also been applied to detect SQL injections, effectively capturing attacker behaviors and enhancing network security measures [24]. Paul et al. [30] introduced SQLR34P3R, a comprehensive detection framework combining risk analysis with CVSS and EPSS scoring, capable of blocking attackers after detecting multiple malicious attempts.

Weak password security has been addressed through various strategies. Pelchen et al. [31] implemented a semi-automatic password-cracking algorithm that dynamically switches attack methods for optimal cracking rates. Hong and Lee [18] proposed a deep learning-based password assessment model suitable for IoT devices, leveraging features like LUDS requirements, zxcvbn scores, and Levenshtein distances to predict password security. Bodkhe et al. [7] used Markov models to improve password complexity evaluation, addressing shortcomings in existing measurement methods. Reddy and Neetha [33] proposed a multifaceted approach to strengthen password security, incorporating strong password policies, multifactor authentication, and user education.

Unauthorized access vulnerabilities have been extensively studied. Omotunde and Ahmed [29] reviewed traditional access control mechanisms like DAC, MAC, and RBAC, highlighting their strengths and limitations. Shi et al. [34] proposed the token-constrained permission delegation algorithm (TCPDA) for blockchain-based access control, ensuring dynamic permission delegation and reducing unauthorized access risks. Razaque et al. [32] developed the hybrid unauthorized data handling (HUDH) scheme, integrating encryption, access control, and intrusion detection to enhance cloud security. Li et al. [22] introduced a global flow diagram to monitor cross-level access in hybrid cloud environments, ensuring the legitimacy of access requests.

Research on token tampering has focused on improving the security of JSON Web Tokens (JWT). Hong et al. [19] proposed an authentication scheme combining JWT and IMEI uniqueness to prevent unauthorized access. Ahmed and Mahmood [1] addressed JWT vulnerabilities by introducing time-based token generation methods, enhancing resistance to attacks. Nugraha et al. [27] compared JWT with PASETO in RESTful APIs, revealing performance and security tradeoffs. Alkhulaifi and El-Alfy [2] evaluated lattice-based post-quantum cryptographic algorithms like DILITHIUM and qTESLA for JWT security, ensuring resistance to quantum attacks.

Sensitive information disclosure has been mitigated through innovative cryptographic techniques. Celik et al. [9] proposed a Shamir's secret sharing-based key management scheme, enhancing secure key access in cloud environments. Gan et al. [15] introduced a revocable key aggregation cryptosystem (RKAC) to allow dynamic control of access rights during data-sharing processes. Dhinakaran et al. [13] integrated quantum key distribution (QKD), CRYSTALS-Kyber, and zero-knowledge proofs (ZKPs) to safeguard cloud systems against quantum computing threats. Vignesh et al. [36] proposed a secure k -nearest neighbor (k -NN) query scheme on encrypted cloud data, combining DT-PKC and secure computation protocols for data privacy and efficient querying.

III. LLM-EDUATTACKGRAPH

LLM-EduAttackGraph operates in two main stages. First, we perform comprehensive website fingerprinting using our custom-designed *Website Fingerprinting* module, which scans the target website and collects key structural and component-level information. The results are aggregated into a structured CSV report that reflects the site's technical footprint. In the second stage, this CSV report is fed into the LLM-EduAttackGraph engine, where an LLM is used to analyze the fingerprint data, infer potential penetration paths, and identify likely vulnerabilities based on historical patterns and learned security knowledge. Section III provide a detailed explanation of each of these two core components.

A. Website Fingerprinting Module

In Fig. 2, the fingerprinting process begins with a hybrid collection approach that combines active probing with passive monitoring to extract detailed network-level characteristics. Leveraging techniques such as TCP/IP protocol stack analysis

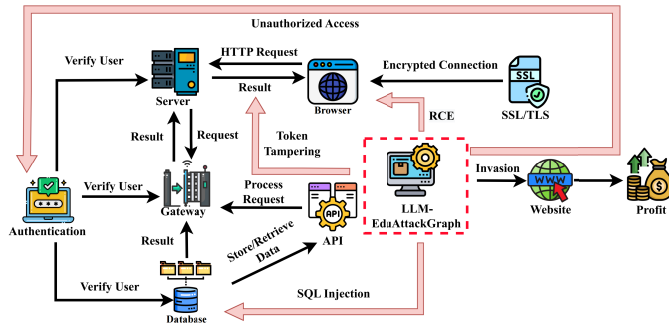


Fig. 1. LLM-EduAttackGraph architecture.

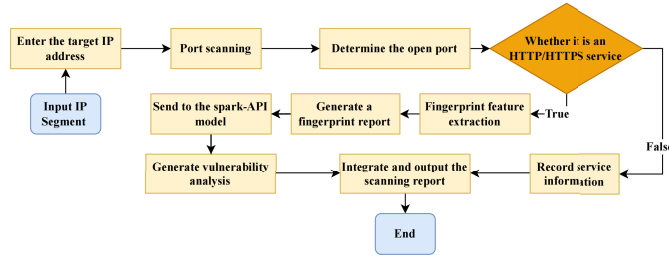


Fig. 2. Workflow of fingerprinting procedure.

and service version fingerprint matching, the system performs in-depth identification of target network services. The collected fingerprint data undergoes standardized preprocessing and is subsequently fed into **LLM-EduAttackGraph** for automated security risk evaluation.

The resulting assessment report includes multiple dimensions of analysis, such as vulnerability similarity scores, References to historically most similar vulnerability identifiers and descriptions, risk-level assessments, and corresponding remediation suggestions. The overall workflow of the website fingerprinting process is as follows: 1) accept a target IP address as input; 2) perform port scanning to identify open ports; 3) conduct service identification on the detected ports to determine running services and their versions; 4) for HTTP/HTTPS services, extract application-layer fingerprint features to generate a detailed fingerprint report; 5) submit the fingerprint report to the *spark-api* model for vulnerability inference; and 6) integrate the inferred vulnerability results into a comprehensive scanning report.

Algorithm 1 demonstrates the technical details of the network fingerprint scanning technology developed in this study. The algorithm is primarily divided into five functional modules. The port scanning module initiates the process by invoking the port-go.exe tool to conduct active TCP probing on the target IP address or domain name. This step determines the open ports and returns a corresponding list. Subsequently, the service identification module employs the server-go.exe tool to further probe these open ports, aiming to identify the version information of the corresponding services. The web fingerprint analysis module is activated upon the detection of HTTP or HTTPS services. Utilizing the finger.exe tool, it extracts the characteristics of the target web page to discern the type and version of the web application. The vulnerability analysis module leverages the spark-api model to analyze the extracted fingerprint information and identify potential security vulnerabilities. Finally, the report generation module integrates all the

Algorithm 1 Website Fingerprinting Procedure

```

1: Parameters:
2: target_ip ▷Target IP address or domain
3: port_scanner = "port-go.exe" ▷TCP/IP stack fingerprinting tool
4: service_scanner = "server-go.exe" ▷Service version identification
5: fingerprint_tool = "finger.exe" ▷Web application fingerprint analyzer
6: ai_model = "spark-api" ▷Vulnerability analysis API
7: procedure SCAN_TARGET(target_ip)
8:   open_ports ← PORT_SCAN(target_ip, port_scanner) ▷Active TCP probing
9:   service_map ← INIT_MAP
10:  for each port in open_ports do
11:    service ← SERVICE_IDENTIFY(target_ip:port, service_scanner)
12:    service_map[port] ← [port, service, "-", "-", ""]
13:    if service ∈ {"http", "https"} then
14:      url ← BUILD_URL(service, target_ip, port)
15:      fingerprint ← WEB_FINGERPRINT(url, fingerprint_tool) ▷Passive analysis
16:      service_map[port] [2] ← url
17:      service_map[port] [3] ← fingerprint
18:      if fingerprint ≠ null then
19:        vulnerability ← AI_ANALYZE(fingerprint, ai_model)
20:        service_map[port] [4] ← vulnerability
21:      end if
22:    end if
23:  end for
24:  GENERATE_REPORT(target_ip, service_map)
25: end procedure
26: function PORT_SCAN(ip, scanner)
27:   raw_data ← EXECUTE(scanner, ["-d", ip])
28:   ports ← EXTRACT_BRACKET_CONTENT(raw_data)
29:   return ports
30: end function
31: function WEB_FINGERPRINT(url, tool)
32:   response ← EXECUTE(tool, ["-u", url])
33:   features ← EXTRACT_BRACKET_CONTENT(response)
34:   if features ≠ ∅ then
35:     return features[0] ▷Primary fingerprint
36:   end if
37:   return null
38: end function

```

acquired information to generate a comprehensive scanning report that encompasses open ports, service versions, website fingerprints, and identified vulnerabilities. This sequential process commences with foundational port scanning, progressively delving into service identification, web fingerprint analysis, and vulnerability assessment, ultimately culminating in the production of a thorough network scanning report.

LLM-EduAttackGraph leverages the fingerprint recognition report to analyze the characteristics of the target website. Based on such feature analysis, LLM-EduAttackGraph

provides accurate target identification and vulnerability scanning paths for penetration testing, ensuring stability and efficiency even in complex network environments.

B. LLM-Driven Penetration Path Inference

The vulnerability reasoning process in **LLM-EduAttackGraph** follows a structured pipeline that integrates historical vulnerability data, vector-based retrieval, and LLM generation. The workflow of LLM-driven penetration path inference is shown in Fig. 3 as follows.

- 1) Historical vulnerability data are collected from open-source repositories such as Awesome-POC, which provides a diverse set of real-world vulnerability examples.
- 2) These examples are segmented into smaller semantically meaningful text chunks to improve retrieval accuracy.
- 3) Each text chunk is then encoded into vector representations using the **BGE embedding algorithm**, specifically the pretrained `bge-small-zh` model from HuggingFace, which captures the semantic meaning of vulnerability descriptions.
- 4) A user query is submitted to the system as the starting point for the reasoning process.
- 5) The query is similarly embedded into the same vector space using the same embedding model.
- 6) Cosine similarity is computed between the query vector and all stored vulnerability vectors to retrieve the most relevant historical cases. A similarity score threshold (e.g., 0.6) is applied—if the score exceeds this threshold, the query is classified as a potential vulnerability.
- 7) All embedded vectors of vulnerability descriptions are stored in a vector database (e.g., Faiss), enabling efficient similarity-based retrieval at scale.
- 8) The retrieved vulnerability chunks are combined with the user query to form a prompt that provides rich context.
- 9) This prompt is passed to an LLM—specifically, DeepSeek—to generate a comprehensive and informed response.
- 10) Finally, the system outputs a detailed answer, including vulnerability analysis, severity assessment, and practical remediation suggestions. The main steps are as follows.

Algorithm 2 outlines the RAG-based vulnerability detection pipeline. Lines 1–5 define the key parameters, including the historical vulnerability dataset, the embedding model, and the similarity threshold. Lines 6–21 describe the construction of the knowledge base, where historical vulnerability descriptions are chunked, embedded using the BGE model, and stored in the FAISS-based vector database. Lines 22–36 process a user query by embedding the input, computing cosine similarity with stored vectors, and, if the similarity exceeds the threshold, generating a response using an LLM. Lines 37–44 define the embedding and similarity computation functions.

Vector DataBase Construction: In the construction process of LLM-EduAttackGraph, relevant information needs to be obtained from historical knowledge sources first. Among them,

Algorithm 2 RAG-Based Vulnerability Detection Pipeline

```

1: Parameters:
2: history_repo = "Dataset" ▷ {Source repository for historical vulnerability data}
3: embedding_model = "Trained Model" ▷ {Pre-trained text embedding model from HuggingFace}
4: similarity_threshold = 0.6
5: llm_model = "DeepSeek"
6: procedure BUILDKNOWLEDGEBase
7:   raw_vulns ← ∅
8:   raw_vulns ← DOWNLOADFILES(history_repo)
9:   text_chunks(history_repo)
10:  for each vuln_file in raw_vulns do
11:    description ← EXTRACTDESCRIPTION(vuln_file)
12:    chunk ← CREATCHUNK(description)
13:    text_chunks.append(chunk)
14:  end for
15:  for each chunk in text_chunks do
16:    vector ← EMBEDTEXT(chunk, embedding_model)
17:    embeddings.append(vector)
18:  end for
19:  vector_db ← INITFAISSDB()
20:  vector_db.add(embeddings)
21:  return vector_db
22: end procedure
23: procedure PROCESSUSERQUERY(vector_db)
24:  query_vector ← EMBEDTEXT(user_query, embedding_model)
25:  max_similarity ← 0.0
26:  for each vector in vector_db do
27:    similarity ← COSINESIMILARITY(query_vector, vector)
28:    if similarity > max_similarity then
29:      max_similarity ← similarity
30:      top_match ← GETTEXTBYINDEX(index)
31:    end if
32:  end for
33:  if max_similarity ≥ similarity_threshold then
34:    prompt ← COMBINATION(top_match, user_query)
35:    llm_response ← Get_LLM(prompt, llm_model)
36:    answer ← FORMATRESPONSE(llm_response)
37:  else
38:    Return No relevant vulnerabilities were detected
39:  end if
40: end procedure
41: function EMBEDTEXT(text, model)
42:  vector ← model.encode(text)
43:  Return vector
44: end function
45: function COSINESIMILARITY(vecA, vecB)
46:  vector ← model.encode(text)
47:  dot_product ← DOT(vecA, vecB)
48:  normA ← NORM(vecA)
49:  normB ← NORM(vecB)
50:  Return dot_product / (normA * normB)
51: end function

```

<https://github.com/Threekiii/Awesome-POC>, as an important historical vulnerability information repository, provides abundant vulnerability example data. After acquiring the data, text chunking technology is employed to disassemble each vulnerability example into independent text units, forming the basic historical information content. To achieve efficient information retrieval and processing, we utilize the Bert BGE embedding model to perform vectorization conversion on the text

$$T = \{t_1, t_2, \dots, t_n\}$$

$$v_i = \text{BGE}(t_i)$$

where T denotes the set of text units after chunking, and each t_i represents an independent text unit. v_i is the vector representation of t_i , which is a vector of fixed length.

BERT, as a pretrained language model, has learned rich linguistic knowledge and semantic information through unsupervised pretraining on large-scale text data. The Bert BGE embedding model further improves and optimizes on the basis of BERT. Its core principle lies in learning the semantic features of text through a deep neural network, mapping text sequences into a fixed-length vector space, and transforming vulnerability description information into low-dimensional dense vector representations.

Prompt Construction: The user input query, serving as the initial trigger for the entire system's operation, must likewise undergo transformation into a vector form via the Bert BGE embedding model. This vectorization process ensures that the user query vector resides within the same semantic vector space as the text block vectors stored in the system

$$T_2 = \{t_1, t_2, \dots, t_m\}$$

$$v_j = \text{BGE}(t_j)$$

In the stage of similarity measurement, the system employs the classic cosine-similarity algorithm for computation. This algorithm quantifies the directional similarity of vectors in high-dimensional space by calculating the cosine value of the angle between two vectors. A value closer to 1 indicates a higher degree of similarity between vectors. Specifically, the system performs pairwise computation between the user query vector and the text block vectors stored in the vector database, comparing their respective vector representations

$$\text{sim}(v_i, v_j) = \frac{v_i \cdot v_j}{\|v_i\| \cdot \|v_j\|} = \cos \theta \in [-1, 1].$$

Then, we set a similarity threshold of 0.6 as the decision criterion. If the computed similarity score exceeds this threshold, the current user query is determined to involve vulnerability-related content. Conversely, if the score is below the threshold, the query is deemed unrelated to vulnerabilities, and a response of *No relevant vulnerabilities were detected* is returned. The determination of a similarity threshold of 0.6 is primarily based on practical experience in the field of information retrieval and a preliminary evaluation of the specific semantic space of this project. In the vulnerability text matching task, this value serves as an empirical tradeoff point for balancing precision and recall. With reference to the range of 0.5–0.7 commonly adopted in the industry for semantic

similarity matching, a threshold of 0.6 has been proven to effectively adapt to the vector distribution characteristics of the knowledge base in this system, thus ensuring that the retrieval results possess both relevance and coverage

Relevant

$$= \begin{cases} \text{vulnerability,} & \text{if } \max_i \text{sim}(v_i, v_j) \geq \text{threshold} \\ \text{Others,} & \text{otherwise} \end{cases}$$

where symbol $\max_i$ denotes the supremum of the pairwise cosine-similarity scores between the target vector v_i and every other vector v_j in the embedding space. Whenever this supremum surpasses the empirically calibrated threshold, the generated textual fragment is classified as vulnerability-relevant.

This rigorous mechanism for similarity calculation and judgment ensures the accuracy and reliability of the system's classification of user queries, enabling precise identification of vulnerability-related issues for subsequent processing.

Upon retrieving vulnerability instances with a similarity score exceeding the user's query, the system undertakes a comprehensive integration of these retrieved contents with the original query

$$\text{Prompt} = \Phi(T_2) \oplus \Psi(T) \oplus \Gamma$$

where $\Phi(T_2)$ represents the user input content, $\Psi(T)$ is the historical vulnerability text corresponding to the vector with the highest similarity to the input in the vector vulnerability library, and Γ denotes the instruction template.

Using semantic correlation analysis, critical elements such as the retrieved vulnerability descriptions, technical characteristics, and historical solutions are structurally reorganized in alignment with the problem context of the user's query. This process culminates in the formation of an input text with explicit contextual orientation, which serves to guide the LLM in conducting reasoning grounded in precise domain knowledge of vulnerabilities.

Response: The generated prompt is input into the LLM (primarily DeepSeek). The final response output by the system not only includes a direct answer to the user's question but also integrates the retrieved vulnerability information to provide a comprehensive summary of the vulnerabilities. Additionally, it puts forward professional suggestions on vulnerability remediation and defense strategies.

C. LLM Performance Test

1) *Experimental Configuration:* This experiment did not employ a GPU. The CPU used was an Intel Core i9-13900HX processor with 16 GB of memory and a 1-TB hard drive.

2) *Experimental Results:* To evaluate the vulnerability identification performance of LLM-EduAttackGraph, this study constructed a dataset containing 90 test samples, including 50 vulnerability data (25 Chinese and 25 English samples each) and 40 nonvulnerability data (15 Chinese and 25 English samples). Experimental results show that the model demonstrated exceptional accuracy in the binary classification task, achieving an overall identification accuracy of 98.89%. Only 1

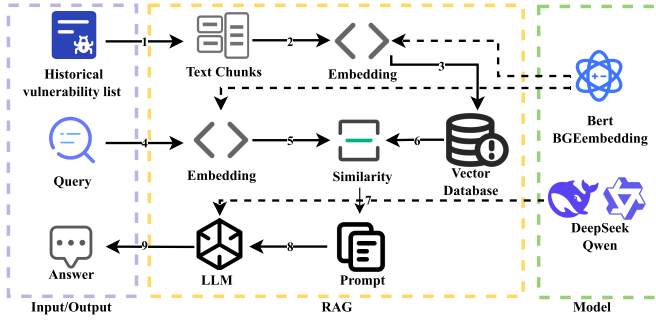


Fig. 3. Workflow of LLM-driven penetration path inference.



Fig. 4. Runtime performance of the LLM. (a) Runtime performance of the LLM in vulnerability data processing. (b) Runtime performance of the LLM in nonvulnerability data processing.

piece of vulnerability data was misclassified as nonvulnerability, while the identification accuracy for nonvulnerability data reached 100%. In terms of operational efficiency, the model exhibited remarkable task adaptability. The search function was only invoked when vulnerability data were successfully identified, with an average processing time of 79.04 s; processing nonvulnerability data required only 9.29 s, reflecting the capability to dynamically optimize resource allocation according to task complexity. These results demonstrate that the LLM ensures high precision while balancing efficient response under different task scenarios, showcasing strong practical application potential. Fig. 4(a) and (b) shows the time LLM-EduAttackGrpah took to process vulnerable and nonvulnerable data. In the 27th test shown in Fig. 4(a), the runtime had a sharp drop because one vulnerable datum was misjudged as nonvulnerable. Except for this, LLM's overall performance in data processing was quite stable.

IV. LLM-EDUATTACKGRAPH FOR VULNERABILITY DETECTION AND ANALYSIS IN EDUCATIONAL WEBSITES

With the assistance of **LLM-EduAttackGraph**, we conducted a systematic penetration testing campaign targeting six major categories of common vulnerabilities in educational websites: ① remote command/code execution; ② SQL injection; ③ weak passwords; ④ unauthorized access; ⑤ token tampering; and ⑥ sensitive information disclosure. As shown in Fig. 5, a heatmap is used to visualize the total number of vulnerabilities and highlight the types of vulnerabilities with the highest occurrence frequencies. In the following sections, we analyze the internal mechanisms behind each type of vulnerability and describe in detail how they were discovered using our tool-assisted approach. For each category, we also propose corresponding mitigation strategies aimed at improving the overall security posture of educational platforms. By deepening the understanding of how these vulnerabilities emerge and are exploited, our goal is to raise cybersecurity awareness and contribute to building a safer and more resilient online environment for educational institutions.

A. Remote Command/Code Execution Vulnerability

We have identified three main causes of RCE vulnerabilities in educational websites: 1) flaws in the framework itself; 2) leakage of sensitive information; and 3) bypassing security checks during file upload. Specific vulnerability details and related information have been presented in a comprehensive manner and described in Table II. In terms of framework vulnerabilities, we have found that multiple commonly used frameworks have known security issues. For example, when key-value pair formatted data such as `user: admin, password: 123456` are discovered in the POST request packet, and if a vulnerable version of Fastjson (Version <1.2.85) is used to process the POST request data, attackers can trigger a domain name system (DNS) request during the deserialization process by constructing malicious JSON data, thereby proving the existence of vulnerabilities in the target system. This can lead to remote command execution and obtaining server permissions. Similarly, frameworks such as Log4j, Struts2, and Shiro also have known vulnerabilities. Attackers can exploit these vulnerabilities to bypass security mechanisms and execute arbitrary code to control the server by constructing specific requests. The root cause of these issues lies in the security vulnerabilities of the frameworks themselves, which allow attackers to easily obtain server permissions and pose a serious threat to the website. In terms of code vulnerabilities, educational websites also have some serious issues. For instance, the leakage of sensitive information is a problem. Website code may disclose critical sensitive information such as access key (AK) and secret key (SK). Attackers can take over cloud services and use this sensitive information to execute malicious code, thereby directly obtaining server permissions. Additionally, the file upload function also has security risks. Attackers can upload malicious scripts (such as webshells) by using 0 day (unpublished vulnerabilities), *n* day (published but unpatched vulnerabilities) or conventional logic bypass methods, thus controlling the server. The root cause

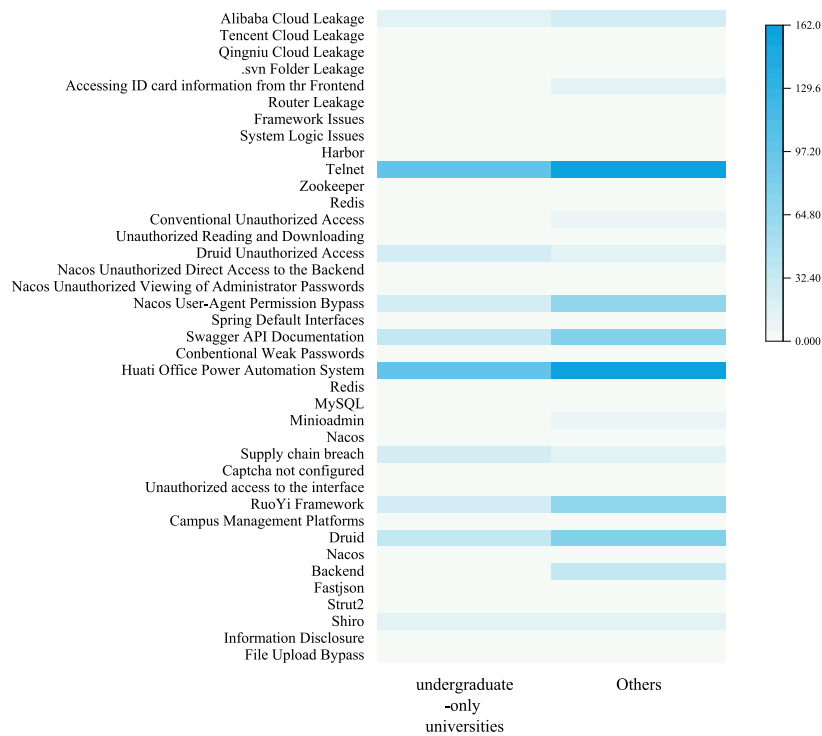


Fig. 5. Heatmap illustrates the spatial distribution and occurrence count of all recorded attacks.

TABLE II
RCE

Reason	Discovery Process		Vulnerability Principle	Impact	Remediation	Quantity
Framework Issues	Fastjson	POST requests containing key-value pairs. Construct malicious JSON data to verify if there are any vulnerabilities.	The backend code uses a vulnerable version of fastjson for deserializing POST data.	Obtain server access.	1.Add WAF to intercept malicious requests. 2.Replace the version of fastjson.	Undergraduate college: 2 Others: 0
	Strut2	Scanners initiate active requests for discovery.	Exploiting known Struts vulnerabilities for attacks.	Obtain server access.	Update framework versions.	Undergraduate college: 0 Others: 2
	Shiro	Scanners initiate active requests for discovery.	Exploiting known Shiro vulnerabilities for attacks.	Obtain server access.	Update framework versions.	Undergraduate college: 12 Others: 14
Information Disclosure	AK,SK Leakage	Found leaked AK,SK.	Achieve code execution by taking over the cloud.	Obtain server access.	Remove leaked AK and SK.	Undergraduate college: 0 Others: 4
File Upload Bypass	At file upload, attempt nday, 0day, and conventional logic bypasses.		Failing to effectively filter malicious script code uploads.	Obtain server access.	Repair code logic issues.	Undergraduate college: 3 Others: 0

TABLE III
SQL INJECTION

Reason	Discovery Process		Vulnerability Principle	Impact	Remediation	Quantity
Framework Issues	Nacos	Test	Injection points	The database accounts and passwords has been leaked.	Upgrade to the latest version.	Undergraduate college: 0 Others: 2
		<i>Nacos/v1/cs/ops/derby?sql =select* from users.</i>	may allow malicious SQL statements to execute.			
Conventional SQL Injection	Backend	After logging into the backend, perform SQL testing on certain interfaces.	The system does not filter parameters when receiving them. Malicious SQL statements can be executed.	The database accounts and password has been leaked.	Add filtering or preprocessing, etc.	Undergraduate college: 2 Others: 35

of these issues is the insufficient consideration of security during code writing, which allows attackers to bypass security mechanisms and perform malicious operations to obtain server permissions.

In general, only 37 RCE vulnerabilities were found in educational websites, accounting for a small proportion (approximately 3.85%) of the vulnerabilities identified in educational websites. However, the existence of vulnerabilities indicates that using outdated versions of frameworks can increase the risk of websites being attacked. Therefore, timely updating and patching these frameworks to their latest versions is crucial for maintaining website security.

In response to the aforementioned RCE vulnerabilities, we need to take a series of measures for remediation. For framework vulnerabilities, the most effective solution is to update the framework versions in a timely manner. Updating frameworks such as Fastjson, Log4j, Struts2, and Shiro to the latest versions or adopting published patch methods can effectively prevent attackers from exploiting known vulnerabilities. Second, for code vulnerabilities, we need to enhance the security review of the code. Removing leaked sensitive information ensures that it will not be obtained by attackers. Strengthening the filtering of file types and content can prevent attackers from uploading malicious scripts. Addition-

ally, adding a web application firewall (WAF) can intercept malicious requests and further improve the security of the website.

B. SQL Injection Vulnerability

A total of 39 SQL injection vulnerabilities were revealed in this experiment. The identified vulnerabilities were categorized into two types: 1) SQL injection vulnerabilities caused by framework issues and 2) conventional SQL injection vulnerabilities. Specific vulnerability details and related information have been comprehensively presented and described in Table III. Nacos is an open-source platform from Alibaba, designed to help users develop applications. For example, there is a vulnerability in the Nacos framework. By using the test statement `Nacos/v1/cs/ops/derby?sql = select * from users`, it can be found that users have control over the data queried by SQL statements and there is an injection point, allowing malicious SQL statements to be executed. This can lead to the leakage of database accounts and passwords, malicious manipulation of the database, and even privilege escalation to obtain shell access. Conventional SQL injection issues mainly occur in the backend category. After logging into the backend, SQL injection was discovered when conducting SQL tests on certain interfaces. The principle of vulnerability is that malicious SQL statements are constructed for specific parameters, and the system passes parameters without filtering them, resulting in the parameters being concatenated and executed as part of the system SQL. This can similarly lead to the leakage of database accounts and passwords, malicious manipulation of the database, and even privilege escalation to obtain shell access.

Whether it is a framework issue or a conventional SQL injection issue, the solution can be to upgrade to the latest version. For framework issues, timely upgrades to the latest version can effectively prevent the exploitation of known vulnerabilities. For conventional SQL injection issues, although upgrading the framework version may not directly fix code-level vulnerabilities, combining it with code-level protection measures, such as adding parameter filtering and using prepared statements, can effectively prevent the execution of malicious SQL statements. Through these measures, the risk of website attacks can be significantly reduced, and the security of the database and server can be protected.

SQL injection vulnerabilities account for 4.05% of the total vulnerabilities. It is logical that the security vulnerabilities that cause SQL injection in educational websites are similar in nature to those causing RCE vulnerabilities. When attackers attempt to execute commands remotely, they are likely to target the database to illicitly obtain sensitive information. The commonality of these attack methods lies in the fact that both SQL injection and RCE involve attackers exploiting security flaws in the system or application to achieve unauthorized access and manipulation of the backend database. Therefore, understanding the similar causes of these two types of vulnerabilities is crucial for developing effective defense strategies. We call upon the website administrators of relevant educational institutions to continuously monitor and promptly

upgrade server components to guard against potential risks of information leakage. Maintaining the latest status of system components can significantly reduce the likelihood of security vulnerabilities being exploited, thereby protecting the data security of the institution and its users.

C. Weak Password

In this experiment, 535 security vulnerabilities related to weak passwords and default passwords were identified, accounting for 55.61% of the total identified vulnerabilities. The commonality of these issues is that attackers can gain access to the backend system through guessing, brute forcing, or exploiting default account credentials, thereby obtaining sensitive information, controlling the server, and even expanding the scope of the attack. Table IV systematically organizes and summarizes the specific details and related information of these vulnerabilities. The backend login pages have obvious security vulnerabilities, the root cause of which is that the system's default accounts and passwords are too simple, and effective access control measures have not been implemented, allowing attackers to easily obtain backend access rights. For example, open-source components such as Druid, a high-performance database system for real-time analysis. MinIO, an open-source object storage server. MySQL, an open-source relational database management system and Redis, an open-source high-performance key-value storage database, are widely used in web development. The default login page of the Druid component can be tested by accessing a specific path. If Druid uses the default password `royal 123456` or other simple passwords, attackers can easily log in to the backend and obtain sensitive information such as sessions. When components like MinIO, MySQL, and Redis are deployed on ports and those ports are not closed, targeted attacks can be launched against these ports. MySQL runs on port 3306, while Redis runs on port 6379. Attackers can utilize scripts to conduct fuzz testing based on weak passwords for these ports, attempting to brute force the default passwords of the components.

The backend of the smart campus management platform has issues with empty accounts and weak passwords. Attackers can discover default passwords through brute forcing and then log in to the backend to obtain sensitive data such as student information. The Ruoyi framework, as an open-source Java development platform, can help users quickly develop websites. However, it is this widely used framework whose backend login page has been found to have a default weak password vulnerability. Similarly, the Huatian Power Office Automation System (Huatian Power OA), as an office automation software, also exposes default password issues on the backend login page. Attackers can use these default passwords to log in to the backend, modify student information, leak sensitive data, and even control equipment within the campus.

In website development, developers often rely on existing open-source frameworks. Some educational websites, especially schools, are built and maintained by their internal teachers. However, due to the lack of experience of these internal technical personnel in website construction, it may lead to the failure to change default passwords in a timely manner during website deployment, thereby introducing security risks. In

TABLE IV
WEAK PASSWORD

Reason	Discovery Process	Vulnerability Principle	Impact	Remediation	Quantity
Druid	Access druid/login.html and other Druid paths to test for login page and weak passwords.	Druid has default username and password.	Druid backend information leak.	1. Restrict Druid access to only internal network. 2. Change the default password.	Undergraduate college: 37 Others: 78
Campus Management Platforms	Black-box testing reveals empty accounts, then weak password brute-forcing exposes password.	Discover vulnerabilities by testing weak passwords through black-box penetration testing.	External individuals access the backend to obtain sensitive information.	Allow access system only from the campus intranet.	Undergraduate college: 0 Others: 1
RuoYi Framework	The login form has characteristics of the RuoYi framework.	RuoYi Framework has default username and password.	Modify student information; Control of internal devices, etc.	Allow access system only from the campus intranet.	Undergraduate college: 25 Others: 68
Obtain accounts and passwords through unauthorized access to the interface	Obtain accounts and passwords through unauthorized access vulnerability.	Use other vulnerabilities to obtain account credentials for login.	Modify student information; Control of internal devices, etc.	Fix unauthorized access vulnerabilities.	Undergraduate college: 0 Others: 4
Captcha not configured	Login does not implement captcha verification, attempt to use Burp Suit to intercept and brute-force, observing the response packet length.	Confirm the real accounts and password through a large number of requests.	Modify student information; Control of internal devices, etc.	1. Add captcha. 2. Use phone number short message service verification.	Undergraduate college: 3 Others: 1
Supply chain breach, accounts and password leak	Supply chain breach results in a large amount of school accounts and passwords being leaked.	Expand the attack surface by exploiting the supply chain.	The school information system supplied by the vendor was hacked.	Strengthen the security of the supply chain.	Undergraduate college: 23 Others: 11
Nacos	Test default username and password on Nacos default login page.	Use default username and password.	Access keys exposed.	Change the default username and password.	Undergraduate college: 0 Others: 2
Minioadmin	Test default username and password on Minioadmin default login page.	Use default username and password.	Information and data leakage.	Change the default username and password.	Undergraduate college: 5 Others: 6
MySQL	Port 3306 is open, using a script to perform weak password fuzz testing on port 3306.	Brute-force weak passwords.	Information and data leakage.	Close the external access to port 3306.	Undergraduate college: 4 Others: 1
Redis	Port 6379 is open, using a script to perform weak password fuzz testing on port 6379.	Brute-force weak passwords.	Information and data leakage.	Close the external access to port 6379.	Undergraduate college: 2 Others: 0
Huati Office Power Automation System Weak Passwords	The login page is the Huatian Power OA login page.	Huatian Power OA has default username and password.	Access the backend.	Change the default username and password.	Undergraduate college: 101 Others: 162
Conventional Weak Passwords	The backend has a login form, allowing for weak password testing.	Brute-force weak passwords.	Access the backend.	Change the default username and password.	Undergraduate college: 0 Others: 1

addition to developing and building websites by internal technical personnel, some educational institutions choose to purchase ready-made website functional module solutions from external suppliers, which does not mean complete security. As can be seen in the table, 34 websites were affected by the supply chain compromise. Companies provide built website functionalities for educational institutions and undertake data storage and operation, and maintenance responsibilities. Under this model, once the databases managed by these companies are leaked, the information stored in the universities is also at risk of being leaked. In addition, during the execution of routine weak password testing, we found that 101 undergraduate university websites and 162 other types of educational websites had weak password vulnerabilities. Then, four websites did not have a CAPTCHA set up, and the student accounts of most schools are student IDs, which are also likely to appear in scholarship announcements. Theoretically, as long as there is enough time, we can test the correct accounts and passwords.

During the experimental phase, it was observed that a significant number of vulnerabilities were attributed to weak passwords. The prevalence of weak password vulnerabilities across educational websites is alarmingly high. Consequently, it is imperative for these institutions to enhance their security protocols by enforcing stricter control measures.

D. Unauthorized Access Vulnerability

Table V systematically organizes and summarizes the specific details and related information of unauthorized access

vulnerabilities. A total of 271 unauthorized access vulnerabilities were found in educational websites, accounting for 28.20% of the total identified vulnerabilities. The commonality of these issues is that attackers can access specific interfaces or services without authorization, thereby obtaining sensitive information or performing malicious operations. Whether it is through the leakage of Swagger interface documentation and other configuration issues or through accessing the default paths that exist in the Java development framework Spring, or through constructing malicious requests via the Nacos system's default interface, these issues all indicate that many websites have serious vulnerabilities in the access control of interfaces and services. Attackers can exploit these vulnerabilities to obtain sensitive information such as user information, passwords, configuration information and even perform malicious operations such as creating new users, viewing administrator passwords, and so on. In addition, components such as Redis, Zookeeper, Telnet, and Harbor all expose issues of unauthorized access. Among them, Zookeeper is a service designed for distributed applications, developed by the Apache Software Foundation. Telnet, short for Telecommunication Network, is a network communication protocol. Harbor is developed by VMware to help enterprises manage and deploy containerized applications more efficiently. Attackers can gain unauthorized access by scanning ports or exploiting known vulnerabilities, thereby obtaining sensitive information or carrying out malicious operations. The existence of these issues not only poses a threat to website

TABLE V
UNAUTHORIZED ACCESS VULNERABILITY

Reason	Discovery Process	Vulnerability Principle	Impact	Remediation	Quantity
Swagger API Documentation	Discover Swagger API and test the exposed endpoints.	Construct a malicious request using the API documentation to obtain sensitive information.	This vulnerability allows unauthorized access to the administrator service.	1. Disable access to Swagger documentation after development. 2. Restrict access permissions for API endpoint paths.	Undergraduate college: 12 Others: 1
Spring Default Interfaces	Access the default paths in Spring for exploration. Such as /prod-api/actuator, etc.	Specific paths in the Spring framework for accessing configuration information lack proper access control.	Sensitive information leakage, such as environment configuration.	Restrict access permissions after web development is completed.	Undergraduate college: 45 Others: 57
Nacos User-Agent Permission Bypass	Craft malicious request on Nacos default interface to create new user.	Modify header to perform operations on accounts and passwords.	Access keys exposed.	1.Enable service identity authentication. 2. Enable auth for Nacos registry and config center.	Undergraduate college: 0 Others: 1
Nacos Unauthorized Viewing of Administrator Passwords	Craft malicious request on Nacos default interface to view admin password.	The interface lacks authentication.	Access keys exposed.	Set up authentication.	Undergraduate college: 1 Others: 0
Nacos Unauthorized Direct Access to the Backend	Direct access to Nacos default interface leads to backend access.	The login page has not been set up, resulting in direct access being allowed.	Access keys exposed.	Set up authentication.	Undergraduate college: 0 Others: 1
Druid Unauthorized Access	Access the default paths in Druid, such as /druid/index.html.	The login page has not been set up, resulting in direct access being allowed.	Set up authentication.	Druid backend information leakage.	Undergraduate college: 8 Others: 23
Unauthorized Reading and Downloading	Access URL or craft packet to trigger unauthorized read/download.	The interface lacks authentication.	The impact varies.	Set up authentication.	Undergraduate college: 9 Others: 7
Conventional Unauthorized Access	Unauthorized ops via URL access or malicious packet.	The interface lacks authentication.	The impact varies.	Set up authentication.	Undergraduate college: 47 Others: 46
Redis	Port 6379 is open.	Exploiting known Redis vulnerabilities for attacks.	Obtain server access. Read and modify data.	Disable remote access to port 6379; Don't run Redis as root.	Undergraduate college: 7 Others: 2
Zookeeper	Port 2181 is open.	Exploiting known Zookeeper vulnerabilities for attacks.	Read sensitive information.	Disable remote access to port 2181.	Undergraduate college: 0 Others: 2
Telnet	Port scanning reveals the service as Telnet.	Telnet has no username/password set by admin.	Read sensitive information.	Disable the Telnet service.	Undergraduate college: 1 Others: 0
Harbor	The login page is Harbor.	Perform unauthorized login by exploiting publicly available vulnerabilities.	Mirror database accounts and passwords file leaked.	Change the Harbor version.	Undergraduate college: 0 Others: 1

security but may also lead to serious consequences such as user data leakage and malicious exploitation of systems.

Specifically, regarding the unauthorized access issue of the Swagger interface documentation, attackers can discover interfaces through configuration issues such as the leakage of the Swagger interface documentation and then use the interface documentation to conduct tests. They can construct malicious requests according to the interface documentation to obtain sensitive information, such as user information and passwords. Remediation methods include prohibiting access to the Swagger documentation after development, implementing interface documentation verification, and restricting access to API interface paths. For the unauthorized access issue of Spring's default interfaces, attackers can access the default paths existing in Spring, such as /prod-api/actuator, /prod-api/actuator/env, and /prod-api/actuator/health, to obtain configuration information and application version details. The fix is to restrict access permissions after development and prohibit access to the default Spring interfaces. Regarding the unauthorized access issue of Nacos, if the Nacos system does not strictly validate the User-Agent field when processing user authentication requests, it will treat requests with the User-Agent set to Nacos-Server as internal requests, thereby bypassing permission verification. At this point, attackers are able to dispatch a POST request to the default interface of the Nacos system. The target uniform resource locator (URL) here is <http://your-ip:8848/nacos/v1/auth/users>, and they will also attach the data `username=test1&password=test1` along with the request. Once the request is successful, attackers can create a new user named test1 and log in to the system with the

same password test1. To prevent attackers from creating new users or performing other unauthorized operations through the User-Agent permission bypass vulnerability, the application. The properties configuration file of Nacos can be modified to enable service identity recognition or enable permission authentication for the Nacos registration and configuration center. Additionally, attackers may attempt to construct malicious requests through the default interface of the Nacos system to view administrator passwords or directly access the backend. The Nacos backend typically contains website passwords, database passwords, and so on, and vulnerabilities in the Nacos system can lead to information leakage. Components such as Druid, Redis, Zookeeper, Telnet, and Harbor have omissions in authentication settings. Attackers can obtain Druid backend information, such as session, by accessing the default path of Druid, like /druid/index.html. Alternatively, they can scan the open ports of the website and conduct targeted tests on these ports. For unauthorized vulnerabilities due to the lack of component authentication settings, general remediation methods can include setting up authentication, closing remote access ports, changing component versions, or shutting down the service.

Similar to the causes of weak password vulnerabilities, developers failed to adequately consider website security issues after the completion of website development, resulting in the absence of key security measures such as authentication, thus causing unauthorized access vulnerabilities. It is worth noting that among the 271 unauthorized access vulnerabilities, 102 were due to unauthorized access via default Spring framework interfaces and 93 were due to conventional unauthorized

TABLE VI
TOKEN TAMPERING

Reason	Discovery Process	Vulnerability Principle	Impact	Remediation	Quantity	
Framework Issues	Construct JWT token with Nacos default secret key.	Discover JWT credentials in Nacos using default secret key.	Craft malicious JWT token to bypass auth mechanism.	Access keys exposed.	1. Modify the default JWT secret key in Nacos. 2. Restrict access to internal network only.	Undergraduate college: 3 Others: 1
System logic Issues	Retrieve JWT information from the package and try to construct it.	JWT lacks key validation, enabling malicious JWT token construction for various operations.	The impact varies.	1. Encrypt JWT. 2. Validate JWT construction 3. Apply key encryption to JWT.	Undergraduate college: 1 Others: 2	

access. Unauthorized access vulnerabilities are usually caused by improper system configuration of file access permissions or a lack of effective access control measures. The Spring framework has certain paths that can access configuration information. After some educational websites were developed, developers failed to set up proper access control or did not protect the Spring access paths, resulting in unauthorized access vulnerabilities. To mitigate these vulnerabilities, it is recommended to take measures, including but not limited to strengthening input validation, implementing strict permission checks, regularly auditing system security configurations, and updating security policies.

In educational websites, unauthorized access vulnerabilities and weak password vulnerabilities have the highest occurrence frequency. We observed that unauthorized access vulnerabilities and weak password vulnerabilities primarily emerged in the peripheral assets of schools (e.g., websites whose domain names do not belong to the university but are marked with the copyright ownership of a certain university and store student data). The peripheral assets of educational institutions are prone to having weak passwords such as *admin*, *admin123*, and 123456.

E. Token Tampering

Table VI summarizes and consolidates the process of discovering token tampering vulnerabilities, their principles, the impacts they cause, as well as the methods to fix them. In this experiment, only seven token tampering vulnerabilities were identified, accounting for 0.73% of the identified vulnerabilities in educational websites. Token tampering mainly focuses on the use of Nacos system and JWT tokens. The common point of these vulnerabilities is that attackers can bypass permission checks by constructing malicious JWT tokens, thus obtaining sensitive information or carrying out malicious operations. Attackers can exploit these vulnerabilities to bypass permission check mechanisms to obtain sensitive information, such as database passwords, and even carry out a variety of malicious operations.

In the Nacos framework, the default key of Nacos can be used to generate JWT tokens. If the developer does not modify the default key of Nacos or does not check the key, attackers can try to construct authentication tokens and bypass the authentication mechanism by exploiting the default configured key in the Nacos platform and common account information, thus directly obtaining access to Nacos. Modifying the default key of Nacos or setting Nacos to be accessible only within the internal network can effectively avoid the risks brought by token tampering vulnerabilities. Another type is due to the vulnerability at the system logic level. When the system

processes data packets, it will return JWT information to the client. Attackers can take advantage of this to try to construct malicious JWT tokens and bypass the system's security mechanisms. The impact range of JWT vulnerabilities caused by system logic vulnerabilities is relatively wide, and the specific impact depends on the system's specific implementation and configuration. Checking whether the JWT construction complies with the standard, re-encrypting the JWT, or encrypting the JWT key can enhance the system's defense against token tampering vulnerabilities.

F. Sensitive Information Disclosure

Table VII provides a systematic organization and summary of the specific details and relevant information of these vulnerabilities. A total of 57 instances of sensitive information leakage were found in educational websites, accounting for 5.93% of the identified vulnerabilities. There are four main causes of sensitive information leakage in educational websites: cloud key leakage, svn folder leakage, access to ID card information on the front end, and router accounts and passwords leakage. Among them, the svn folder is a hidden folder created by the Subversion version control system in every folder under version control, used to store version control information related to that folder. The leakage of cloud keys is the main factor leading to sensitive information exposure. In the Alibaba Cloud environment alone, we found 35 incidents related to AK and SK leakage. The leakage of AK and SK may be caused by various factors such as developers' negligence and improper configuration files and is not entirely due to issues with the cloud server providers. These issues share a common point that attackers can obtain sensitive information through various means and then use this information for malicious operations, posing a serious threat to the security of websites and the confidentiality of user data.

Leakage of AK and SK from Alibaba Cloud, Tencent Cloud, and Qingniu Cloud has been identified. In the actual process of website development and maintenance, many developers fail to properly manage sensitive information when using cloud services. AK and SK, which should be strictly controlled, are exposed, and the JavaScript (JS) content does not delete these sensitive details. Anyone who can access the JS file may easily obtain these critical keys. Meanwhile, there is also a risk of information leakage through accessing the website source code and backend systems. If sensitive information is not hidden in the source code, attackers can use these clues to further exploit other system vulnerabilities or directly use the sensitive information for malicious attacks. With the leaked AK and SK, attackers can directly access cloud data, obtain shell permissions of the website by writing malicious scripts, and even

TABLE VII
SENSITIVE INFORMATION DISCLOSURE

Reason	Discovery Process	Vulnerability Principle	Impact	Remediation	Quantity
Alibaba Cloud AK and SK leakage.	Exposed AK and SK in JS file; Leakage via website source code; Leakage after backen access.	Systems use AK/SK to handle files but improper permission settings.	Use AK and SK to access cloud data; Script-based getshell, potentially compromising many websites.	1. Delete exposed AK and SK. 2. Shutdown cloud server. 3. Modify AK and SK information.	Undergraduate college: 12 Others: 23
Cloud key leakage	Tencent Cloud AK and SK leakage.	Exposed AK and SK in JS file; Leakage via website source code; Leakage after backen access.	Systems use AK/SK to handle files but improper permission settings.	1. Delete exposed AK and SK. 2. Shutdown cloud server. 3. Modify AK and SK information.	Undergraduate college: 3 Others: 2
	Qingniu Cloud AK and SK leakage.	Exposed AK and SK in JS file; Leakage via website source code; Leakage after backen access.	Systems use AK/SK to handle files but improper permission settings.	1. Delete exposed AK and SK. 2. Shutdown cloud server. 3. Modify AK and SK information.	Undergraduate college: 2 Others: 0
.svn folder leakage	Accessing the .svn path on the website; if data is returned, a vulnerability may exist.	Forget to delete .svn files after development.	Leak sensitive information such as source code, database credentials, etc.	Delete the .svn path after coding is complete.	Undergraduate college: 1 Others: 0
Accessing ID card information from the frontend	Student real data is commented in the frontend code.	ID card information appears in places with no access control.	Sensitive information leakage.	Delete the data exposed on the frontend.	Undergraduate college: 0 Others: 12
Router username and password leakage	The sensitive information is present in the JS code.	Some school routers have vulnerabilities, and passwords are in the frontend code.	Facilitate internal network intrusion.	Delete router username and password information.	Undergraduate college: 0 Others: 2

execute remote code to enter the internal network environment. Shutting down the cloud server or modifying the AK and SK information can prevent attackers from obtaining cloud data or performing other malicious operations through the leaked keys. In addition to the leakage of cloud key information, some websites have management vulnerabilities. For example, improper exposure of ID card information on the front end, .svn sensitive information leakage, and leakage of router accounts and passwords. During website development, code management should be strengthened to ensure that all code is properly cleaned up after completion, avoiding unnecessary sensitive information such as residual .svn paths. The content displayed on the front end should be strictly reviewed, and any potentially leaked sensitive data should be deleted along with the timely clearance of router-related configuration information.

V. DISCUSSION

Currently, the ability of automated tools to detect logical flaws is generally not as effective as manual detection. LLM-EduAttackGraph does not yet support the detection of logical flaws. However, during our team's security testing of educational websites, we successfully identified logical flaw vulnerabilities in six undergraduate university websites and nine other types of educational websites based on the information crawled by LLM-EduAttackGraph. In our sample, we found that key assets of educational websites, typically composed of the school's name followed by domain extensions like .edu or .org, and usually the official websites of the schools, are more prone to logical vulnerabilities, while the frequency of conventional Top ten vulnerabilities on these official websites is comparatively lower than other educational websites. Additionally, it is particularly important to note that in the sample of educational websites analyzed in this study, we identified one site that exhibited signs of being hacked. Upon reviewing the site's JS code, we discovered suspicious page links unrelated to the site that could trigger page redirects when clicked. When accessing the site, a network load presented an unauthorized black page. Therefore, we concluded that the site might have been compromised by hackers. The site can be remediated by: 1) removing the

malicious links embedded in the JS code; 2) clearing all unauthorized black pages; and 3) deleting all unnecessary logical jumps in the JS. This particular case of a suspected hacked educational website reveals the risks that educational sites currently face. It also serves as a reminder to educational administrators and technical staff to strengthen their focus and investment in cybersecurity to ensure the safety of educational information and the stability of the educational environment.

In the process of advancing society as a whole toward intelligence and modernization, the Internet has played an essential bridging role. Therefore, conducting cybersecurity research and ensuring the security of information flow is an indispensable part of promoting social development. Educational websites have numerous systems, a large volume of data, and a wide range of influence, necessitating implementing cybersecurity measures and data protection governance. Moreover, educational websites store a vast amount of personal information, with student data making up the majority. Protecting educational websites from attacks by malicious individuals and ensuring information security has a profound impact on protecting students, especially those who are inexperienced in the ways of the world and underage students. Analyzing the specific vulnerabilities of educational websites can help us intuitively perceive the potential security risks within the sites, identify the nature and causes of the vulnerabilities, and assess their potential impact on website security and data integrity. This article differs in focus from previous research in the field of educational website security. While previous studies often concentrated on discussing the social factors behind identified vulnerabilities, this research proactively explores potential specific vulnerabilities within educational websites. By analyzing the details of specific vulnerabilities, including the discovery process, the causes of the vulnerabilities, the specific impacts on educational websites, and providing remediation methods, this study offers new insights into the security vulnerabilities of educational websites from a technical perspective. This methodological shift allows us to gain a deeper understanding of the technical security risks and provides an empirical basis for the preven-

tion and remediation of vulnerabilities. We conducted real tests on hundreds of educational websites within China, and the cybersecurity issues revealed are significantly representative.

A. Universal Applicability and Cross-Regional Validity of Llm-EduAttackGraph

The cybersecurity challenges faced by educational websites are not unique to China. Educational platforms worldwide are continuously encountering various cyber threats and attacks. Even prestigious institutions like Stanford University [10] have been targeted multiple times. Many countries have begun strengthening cybersecurity protections within the education sector, encouraging schools and universities to implement stricter cybersecurity policies and technical measures. Chinese higher education institutions cooperate more closely with domestic enterprises. Many universities can gain practical experience from real cybersecurity incidents through cooperation with large Internet companies (such as 360 Security Technology). Therefore, the analysis of the security of educational websites in mainland China can still offer insights into global cybersecurity efforts and provide valuable experience for cybersecurity management worldwide. Below are some recommendations we propose based on the research.

Although the proposed method was developed and validated primarily using educational websites from Mainland China, its technical framework and implementation mechanisms exhibit strong generalizability and cross-regional applicability. The vulnerability knowledge base (e.g., Awesome-POC) is sourced from globally available vulnerability repositories, covering multiple languages and technology stacks, and is not dependent on the network infrastructure or policy environment of any specific country or region. The fingerprint scanning module is based on standard network protocols and common web technology stacks (e.g., CMS, middleware, and front-end frameworks), enabling it to identify internationally prevalent educational platforms such as Moodle, Blackboard, and Canvas. Moreover, the pretrained language models used (e.g., BGE and DeepSeek) support multilingual understanding and generation and are capable of processing vulnerability descriptions and remediation suggestions in languages including English and Chinese. Therefore, with appropriate localization adjustments to fingerprint rules and prompt templates, the method can be effectively applied to institutions worldwide for network penetration testing and security hardening.

By adopting the RAG framework, LLM-EduAttackGraph enables precise matching and in-depth semantic reasoning between this globally standardized vulnerability knowledge and the real-time fingerprint features of any target website. Consequently, the analytical capability of this method is rooted in the objective mapping of security risks associated with generic IT components, rather than being constrained by regional network configurations or management practices. This attribute ensures that the method maintains consistent analytical accuracy and practical utility in security assessments of websites across diverse countries and regions—encompassing the education, scientific research, government, and enterprise sectors—thereby offering a universally applicable technical solution for cross-regional cybersecurity risk assessment.

B. Scalability Considerations for Large-Scale Deployment

LLM-EduAttackGraph employs a decoupled architecture of *offline knowledge base construction and online lightweight retrieval*. This design places computationally intensive tasks like vectorization and indexing into the offline phase, effectively avoiding performance bottlenecks during online service. The vector database, built on FAISS, supports efficient similarity search and can be deployed in a multimachine environment for horizontal scaling. The fingerprint scanning module utilizes concurrent port probing and service identification, enabling parallel processing of multiple target IPs. By decoupling the knowledge base, fingerprint scanning, and LLM reasoning, each system component can be independently deployed and scaled, thus efficiently preventing resource contention. Furthermore, the vulnerability knowledge base supports incremental updates; only new vulnerability texts need to be vectorized and merged into the existing index, eliminating the need for a full reconstruction and significantly improving maintenance efficiency in large-scale scenarios. Consequently, LLM-EduAttackGraph is well-suited for large-scale deployment.

In the future, LLM-EduAttackGraph can address the exponential growth of vulnerability data in the education sector by constructing a distributed vector retrieval architecture and achieving the decoupling and independent scaling of knowledge base management, fingerprint collection, and reasoning services through microservice-based refactoring. By introducing container orchestration and load balancing mechanisms, it can effectively resolve the bottleneck in resource scheduling under large-scale deployment, while ensuring analysis accuracy, it can significantly reduce the computational power requirements for centralized inference by integrating model quantization technology and edge computing node deployment. This systematic expansion solution will comprehensively enhance the platform's capabilities in multi-institution collaboration and massive data processing and provide key technical support for building a cybersecurity protection system with wide-area coverage and real-time response.

C. Limitations and Suggestions

The data sample of this study is limited to educational websites within mainland China, primarily due to legal, regulatory, and ethical constraints that restrict penetration testing activities to authorized domestic targets. While this geographical focus imposes certain limitations on the universality of the findings, the security landscape of Chinese educational websites provides valuable insights that can inform global cybersecurity practices. In China, higher education institutions often maintain strong partnerships with domestic technology enterprises and cybersecurity firms (such as Qihoo 360), facilitating the rapid exchange of knowledge and practical experience through real-world incident response and collaborative projects. As such, our focused analysis of Chinese educational websites not only addresses a critical national context but also offers generalizable lessons and strategies for enhancing the cybersecurity of educational systems worldwide. Based on our findings, we present the following recommendations.

- 1) To ensure the stability and security of website systems, it is recommended to regularly check the status of the system components. Using outdated versions of components can lead to hackers exploiting known vulnerabilities, resulting in losses. For instance, we found that components like Fastjson, due to their low versions, had known vulnerabilities that had not been patched. Hiesgen et al. [16] also point out that regarding the Log4j vulnerability, even though patches for the new version of the component have been released, the Log4j vulnerability will remain a significant threat for some time to come.
- 2) Improve the training of website management team members and other relevant personnel to increase their security awareness. Through systematic education and training, they can more effectively address potential security threats during the website development process. For instance, the failure to change default passwords in ready-made frameworks and the default use of student IDs as accounts in campus management systems are manifestations of developers' weak security consciousness.

Currently, cutting-edge technologies—particularly deep learning and machine learning—have shown promising results in the field of cybersecurity. As cited in [24] and [22], researchers have successfully applied these techniques to network attack detection. In the future, it may be possible to leverage deep learning tools for the automated protection of educational websites.

VI. CONCLUSION

In this article, we conducted a large-scale vulnerability detection and analysis on real educational websites within mainland China. Based on an in-depth analysis of the specific identification process and principles of the 961 vulnerabilities found on these websites, we assessed the potential adverse consequences of these vulnerabilities and provided guiding suggestions on how to effectively remediate them. Additionally, this study conducted a comprehensive survey of different types of vulnerabilities discovered, including the latest research findings of other scholars on protecting the network from specific vulnerabilities. Starting from a technical perspective, we analyzed the causes of vulnerabilities and discussed the social factors leading to vulnerabilities in educational websites: the lack of experience of developers within educational institutions or outsourcing website development to companies, resulting in the same vulnerabilities across multiple institutions. Our research work helps to deepen the understanding of potential safety hazards in educational websites, and we are committed to strengthening the cybersecurity protection system.

VII. ETHICS STATEMENT

In practical application, the system strictly adheres to cybersecurity ethics protocols, with all testing activities conducted under explicit authorization from the target educational websites. All 961 vulnerabilities identified through the system have

been reported to the respective institutions and subsequently remediated. The system is exclusively designed to assist educational organizations in enhancing their network security capabilities; any unauthorized penetration testing or malicious exploitation is strictly prohibited.

ACKNOWLEDGMENT

Part of this research was conducted at the Department of Information Technology, Yangtze Delta Region Institute of Tsinghua University, Zhejiang, China. The authors would like to express their sincere appreciation to Langzhu Hong, Yongquan Shi, Yikai Zhou, and Tianlong Lin for their invaluable support and insightful suggestions. Their professional expertise in penetration testing and their provision of technical and data support have substantially contributed to the development of this study.

REFERENCES

- [1] S. Ahmed and Q. Mahmood, "An authentication based scheme for applications using JSON Web token," in *Proc. 22nd Int. Multitopic Conf. (INMIC)*, Nov. 2019, pp. 1–6.
- [2] A. Alkhulaifi and E.-S.-M. El-Alfy, "Exploring lattice-based post-quantum signature for JWT authentication: Review and case study," in *Proc. IEEE 91st Veh. Technol. Conf. (VTC-Spring)*, May 2020, pp. 1–5.
- [3] Anonymous, *Police Have Launched an Investigation Into the Suspected Leak of Information About Nearly 20,000 Students At a University in Zhengzhou*. [Online]. Available: <https://cn-sec.com/archives/3026353.html>
- [4] Anonymous, (2023). *Qut Was Attacked By Cyber Hackers and the Identity Information of Tens of Thousands of Students and Teachers Was Leaked*. [Online]. Available: <https://www.sohu.com/a/637243728121117447>
- [5] S. T. Argaw et al., "Cybersecurity of hospitals: Discussing the challenges and working towards mitigating the risks," *BMC Med. Informat. Decis. Making*, vol. 20, no. 1, pp. 1–10, Dec. 2020.
- [6] N. Bayyapu, "SQL injection attacks and mitigation strategies: The latest comprehension," in *Advances in Cybersecurity Management*. Cham, Switzerland: Springer, 2021, pp. 199–220.
- [7] U. Bodkhe, J. Chaklasiya, P. Shah, S. Tanwar, and M. Vora, "Markov model for password attack prevention," in *Proc. 1st Int. Conf. Comput.*, 2020, pp. 831–843.
- [8] I. Bongiovanni, "The least secure places in the universe? A systematic literature review on information security management in higher education," *Comput. Secur.*, vol. 86, pp. 350–357, Sep. 2019.
- [9] B. Celikbas, I. Celikbilek, and E. Ozdemir, "A higher-level security scheme for key access on cloud computing," *IEEE Access*, vol. 9, pp. 107347–107359, 2021.
- [10] (2024). *Stanford University is Warning of a Data Breach Involving 27,000 People and 430gb of Data Being Stolen*. [Online]. Available: <https://news.qq.com/rain/a/20240314A0710300>
- [11] I. David and A. Gervais, "Multi-agent penetration testing AI for the web," 2025, *arXiv:2508.20816*.
- [12] G. Deng et al., "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur.)*, Aug. 2024, pp. 847–864.
- [13] D. Dhinakaran, D. Selvaraj, N. Dharini, S. Edwin Raja, and C. Sakthi Lakshmi Priya, "Towards a novel privacy-preserving distributed multi-party data outsourcing scheme for cloud computing with quantum key distribution," 2024, *arXiv:2407.18923*.
- [14] D. S. Dolliver, A. K. Ghazi-Tehrani, and K. T. Poorman, "Building a robust cyberthreat profile for institutions of higher education: An empirical analysis of external cyberattacks against a large university's computer network," *Int. J. Law, Crime Justice*, vol. 66, Sep. 2021, Art. no. 100484.
- [15] Q. Gan, X. Wang, and D. Wu, "Revocable key-aggregate cryptosystem for data sharing in cloud," *Secur. Commun. Netw.*, vol. 2017, pp. 1–11, 2017.
- [16] R. Hiesgen, M. Nawrocki, T. C. Schmidt, and M. Wählisch, "The Log4j incident: A comprehensive measurement study of a critical vulnerability," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 6, pp. 5921–5934, Dec. 2024.

- [17] S. Hina and P. D. D. Dominic, "Information security policies' compliance: A perspective for higher education institutions," *J. Comput. Inf. Syst.*, vol. 60, pp. 201–211, Mar. 2020.
- [18] K. H. Hong and B. M. Lee, "A deep learning-based password security evaluation model," *Appl. Sci.*, vol. 12, no. 5, p. 2404, Feb. 2022.
- [19] N. Hong, M. Kim, M.-S. Jun, and J. Kang, "A study on a JWT-based user authentication and API assessment scheme using IMEI in a smart home environment," *Sustainability*, vol. 9, no. 7, p. 1099, Jun. 2017.
- [20] S. Krishnamurthy, "The future of business education: A commentary in the shadow of the COVID-19 pandemic," *J. Bus. Res.*, vol. 117, pp. 1–5, Sep. 2020.
- [21] J. Li, W. Xiao, and C. Zhang, "Data security crisis in universities: Identification of key factors affecting data breach incidents," *Humanities Social Sci. Commun.*, vol. 10, no. 1, pp. 1–18, May 2023.
- [22] N. Li, Y. Dong, T. Che, C. Wang, and Y. Gao, "The legitimacy detection for multilevel hybrid cloud algorithm based data access," in *Proc. IEEE Int. Conf. Softw. Qual. Rel. Secur.-Companion*, Aug. 2015, pp. 169–172.
- [23] Y. Li and Q. Liu, "A comprehensive review study of cyber-attacks and cyber security; emerging trends and recent developments," *Energy Rep.*, vol. 7, pp. 8176–8186, Nov. 2021.
- [24] B. Manjunatha K. A. Shastry, M. Mohan, and Aravind, "Intelligent defense strategies: Machine learning-enhanced sql injection detection and prevention via honeypots," in *Proc. Int. Conf. Intell. Algorithms Comput. Intell. Syst. (IACIS)*, 2024, pp. 1–7.
- [25] (2023). *Tens of Thousands of Teacher and Student Information is Sold Abroad! a University in Jiangxi Has Been Fined 800,000 Yuan*. [Online]. Available: <https://www.thepaper.cn/newsDetailforward24265003>
- [26] D. Muduli et al., "SIDNet: A SQL injection detection network for enhancing cybersecurity," *IEEE Access*, vol. 12, pp. 176511–176526, 2024.
- [27] A. F. Nugraha, H. Kabetta, I. K. S. Buana, and R. B. Hadiprakoso, "Performance and security comparison of json web tokens (JWT) and platform agnostic security tokens (PASETO) on RESTful APIs," in *Proc. IEEE Int. Conf. Cryptogr. Informat., Cybersecurity (ICoCICs)*, Aug. 2023, pp. 15–22.
- [28] D. Omeiza and J. Owusu-Tweneboah, "Web security investigation through penetration tests: A case study of an educational institution portal," 2018, *arXiv:1811.01388*.
- [29] H. Omotunde and M. Ahmed, "A comprehensive review of security measures in database systems: Assessing authentication, access control, and beyond," *Mesopotamian J. CyberSecurity*, vol. 2023, pp. 115–133, Aug. 2023.
- [30] A. Paul, V. Sharma, and O. Olukoya, "SQL injection attack: Detection, prioritization & prevention," *J. Inf. Secur. Appl.*, vol. 85, Sep. 2024, Art. no. 103871.
- [31] C. Pelchen, D. A. Jaeger, F. Cheng, and C. Meinel, "The (persistent) threat of weak passwords: Implementation of a semi-automatic password-cracking algorithm," in *Proc. 15th Int. Conf. Inf. Secur. Pract. Exper.*, Nov. 2019, pp. 464–475.
- [32] A. Razaque, N. Shaldanbayeva, B. Alotaibi, M. Alotaibi, A. Murat, and A. Alotaibi, "Big data handling approach for unauthorized cloud computing access," *Electronics*, vol. 11, no. 1, p. 137, Jan. 2022.
- [33] L. N. Reddy and S. S. Neetha, "Analysis on password attacks and its prevention," *Int. J. Res. Publication Rev.*, vol. 5, no. 3, pp. 4677–4681, Mar. 2024.
- [34] J. Shi, R. Li, and W. Hou, "A mechanism to resolve the unauthorized access vulnerability caused by permission delegation in blockchain-based access control," *IEEE Access*, vol. 8, pp. 156027–156042, 2020.
- [35] E. Sun, J. Han, Y. Li, and C. Huang, "A packet content-oriented remote code execution attack payload detection model," *Future Internet*, vol. 16, no. 7, p. 235, Jul. 2024.
- [36] R. Vignesh, D. Deepa, S. C. Mana, and B. K. Samhitha, "Secure k -NN query on encrypted cloud data with multiple keys," *J. Phys., Conf. Ser.*, vol. 1770, May 2021, Art. no. 012034.
- [37] D. Wu et al., "Cybersecurity for digital manufacturing," *J. Manuf. Syst.*, vol. 48, pp. 3–12, Jul. 2018.
- [38] F. Xiao, Z. Yang, J. Allen, G. Yang, G. Williams, and W. Lee, "Understanding and mitigating remote code execution vulnerabilities in cross-platform ecosystem," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 2975–2988.
- [39] J. Xu et al., "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," 2024, *arXiv:2403.01038*.
- [40] H. L. D. Yanchao. (2020). *Police Have Launched an Investigation Into the Suspected Leak of Information About Nearly 20,000 Students At a University in Zhengzhou*. [Online]. Available: <https://www.thepaper.cn/newsDetailforward7754773>
- [41] Y. Zheng and X. Zhang, "Path sensitive static analysis of web applications for remote code execution vulnerability detection," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, May 2013, pp. 652–661.

Chao Liu (Member, IEEE) received the Ph.D. degree in computer science and electronic engineering from the University of Maryland, Baltimore County, Baltimore, MD, USA, in 2021.

He has published as the first and corresponding author in top-tier journals, such as IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY and IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, and in leading conferences, including DSN, SRDS, and E-VOTE-ID. He has also collaborated with David Chaum; their joint work VoteXX was presented at E-VOTE-ID 2022, one of the top international conferences in electronic voting. His research interests focus on asynchronous Byzantine fault tolerance protocols, consensus algorithms for distributed systems, and blockchains.

Jiaxing Liu received the bachelor's degree from Huaqiao University, Quanzhou, China, in 2024. He is currently pursuing the master's degree with Fuzhou University, Fuzhou, China, affiliated with Fujian Provincial Key Laboratory of Data-Intensive Computing and Intelligent Computing and Information Processing.

He conducts research in the field of network security based on large language models.

Boxi Chen (Student Member, IEEE) is currently pursuing the M.S. degree in big data and Internet of Things with Macao Polytechnic University, Macau.

His research interests include blockchain and privacy protection.

Daxin Zhu is the Dean of the School of Computer Science, Quanzhou Normal University, Quanzhou, China. He has published over 60 academic papers in renowned international journals, such as IEEE TRANSACTIONS ON IMAGE PROCESSING and *Theoretical Computer Science*, with more than 40 of them indexed by SCI and EI. He has led or participated in more than 20 scientific research projects, including Fujian Provincial Natural Fund Project, Key Industry-Academia-Research Projects of Fujian Provincial Education Department, and Quanzhou High-Level Talent Program. He has long been engaged in research on industrial internet applications, artificial intelligence applications, and big data technologies.

Ching-Chun Chang (Member, IEEE) received the Ph.D. degree in computer science from the University of Warwick, Coventry, U.K., in 2019.

He is currently a Project Assistant Professor with the National Institute of Informatics, Chiyoda, Japan. His research interests include artificial intelligence, biometrics, cryptography, cybernetics, cybersecurity, forensics, information theory, mathematical optimization, steganography, and watermarking, with over 100 published articles in international journals and conferences and several best paper awards.

Chin-Chen Chang (Fellow, IEEE) has worked on many different topics in information security, cryptography, and multimedia image processing and published several hundreds of papers in international conferences and journals and over 30 books. He was cited over 50 221 times and has an H-factor of 103 according to Google Scholar. Several well-known concepts and algorithms were adopted in textbooks. He also worked with the National Science Council, Ministry of Technology, Ministry of Education, Ministry of Transportation, Ministry of Economic Affairs, and other government agencies on more than 100 projects and holds 36 patents, including one in U.S. and 21 in China.